Dashboard Endpoints Persistence

PYKV SERVER

A SCALABLE IN MEMORY KEY VALUE STORE WITH PERSISTENCE

Milestone 1

- API Server Development & Core Endpoint Implementation

Our Group

- Hrishikesh Konde [TEAM LEAD]
- Snighda Som
- Rutik Santosh Harbale
- Shweta Pandiyan

Modern Architecture

Backend

FastAPI

Frontend

CSS

Persistence

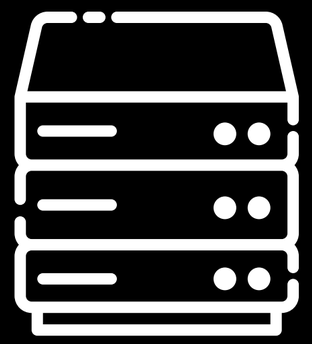
JSON

Zero Loss Workflow

User Submits Data

The interface captures the Key Value pair via a secure web form

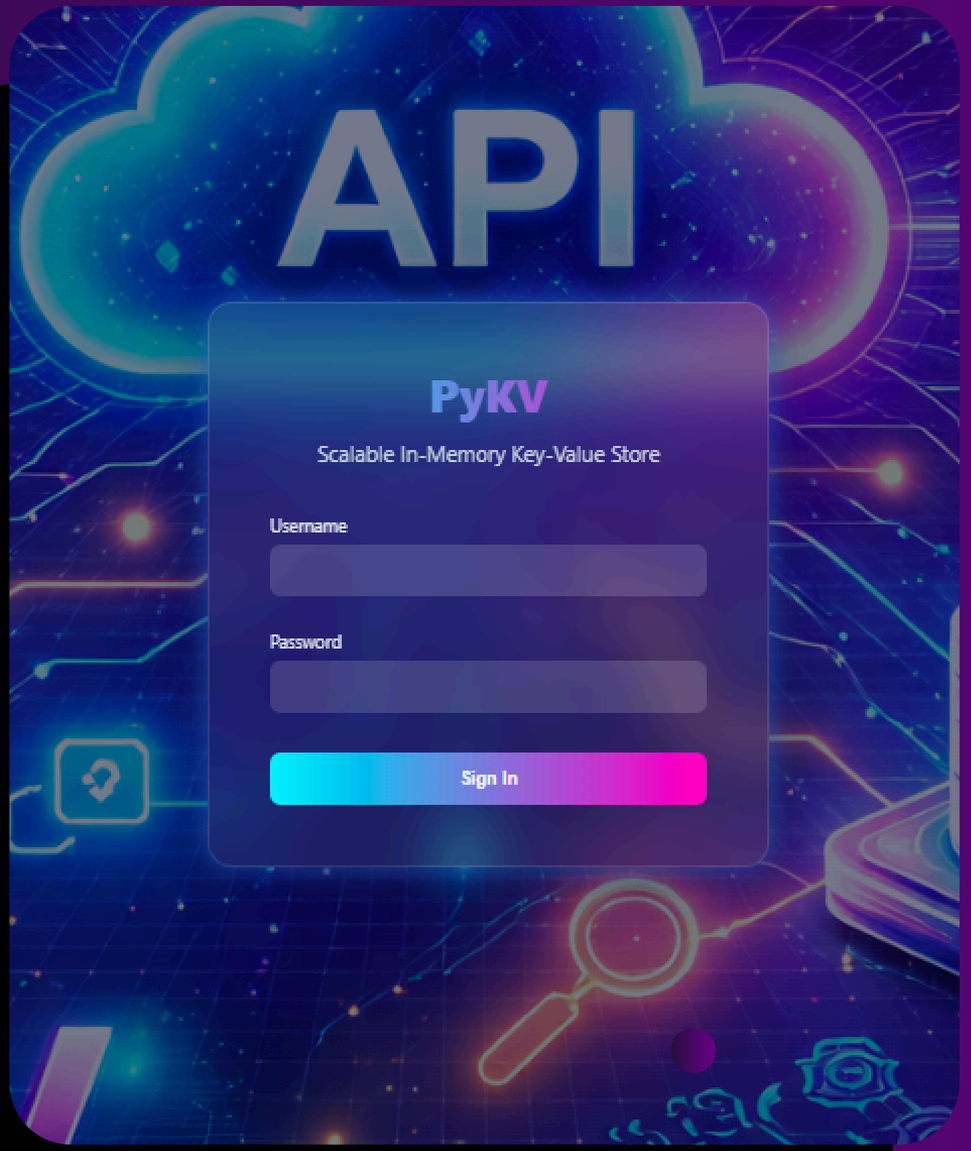
Instant Memory Update



Data is Updated instantly and stored on RAM. CRUD operations are done

Permanent Sync

Every change is automatically mirrored to the data.json vault, ensuring data survives server restarts



The Security Gateway

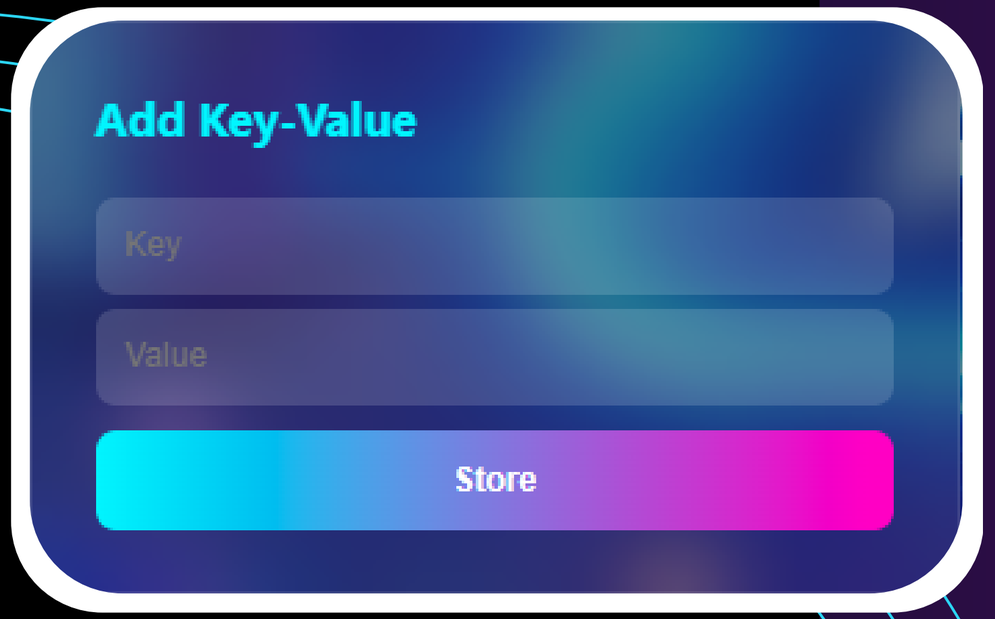
- Authentication

Restricted access using a predefined Administrator credential set (admin / 1234)

- Error Handling

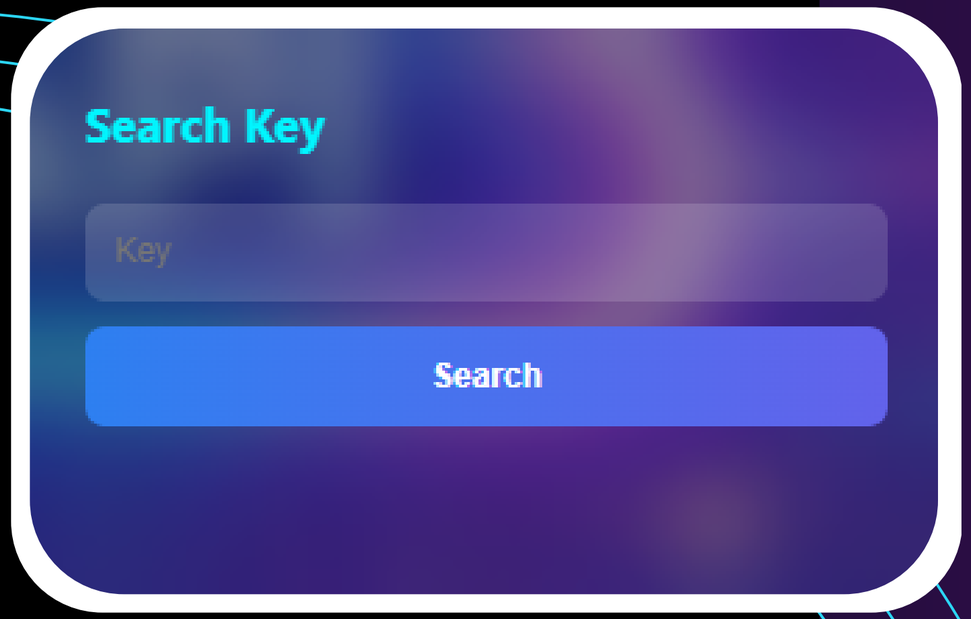
Instant feedback for invalid login attempts via Jinja2 templates

Core Operations



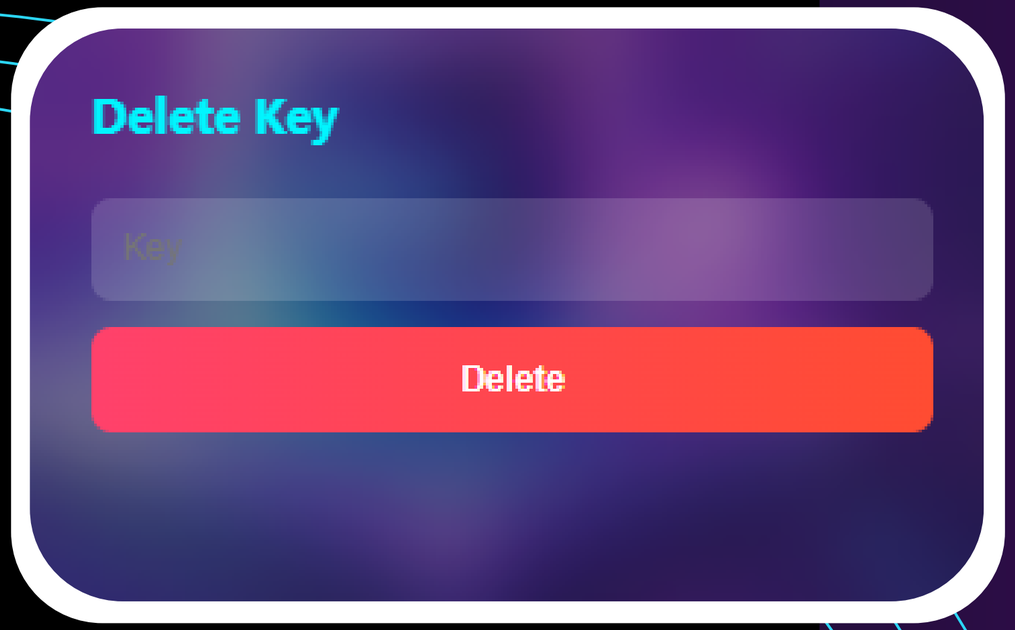
- **Intuitive Input Interface**:
  - The dashboard features a clean, glassmorphism style card dedicated to data ingestion, allowing users to define custom labels (Keys) and their corresponding data (Values).
- **FastAPI Form Handling:**
  - Utilizes high-performance asynchronous POST routes (/add) to capture and process web form data in real-time.
- **Live Memory Injection:**
  - Once a user clicks "Store," the entry is immediately injected into the Python dictionary, enabling sub-millisecond data availability within the application.

Core Operations



- **Targeted Data Discovery:**
  - Provides a dedicated interface for users to perform instant lookups within the storage engine using specific unique identifiers (Keys).
- **High-Speed Response:**
  - Leverages the in memory dictionary architecture to achieve O(1) search complexity, ensuring sub millisecond retrieval of values regardless of store size.
- **Dynamic Result Rendering:**
  - Utilizes Jinja2 templates to display retrieved values directly within the dashboard view, maintaining a seamless user workflow.

Core Operations



- **Targeted Record Removal:**
  - Provides a precise interface for users to purge specific, outdated, or redundant data entries from the storage engine using unique Keys.
- **Synchronized Deletion:**
  - Executes a "two way" delete process removing the entry from the Python in-memory store while simultaneously updating the data.json vault.
- **Asynchronous Processing:**
  - Leverages FastAPI's high speed POST routes (/delete) to ensure the removal happens without causing latency in the user interface.

THANK YOU